

服装爆款趋势分析 MVP 开发文档

1. 功能概述

“服装爆款趋势分析”用于根据用户输入的服装品类、目标人群、使用场景、风格方向，调用 AI 大模型生成一份结构化的爆款趋势分析结果。

用户在前端填写 4 个字段：

```
{
  "category": "品类",
  "target_user": "目标人群",
  "scene": "使用场景",
  "style": "风格方向"
}
```

后端接收参数后，将用户输入组装到固定 Prompt 中，调用 OpenAI 兼容接口，要求 AI 返回标准 JSON。后端保存用户输入、组装后的 Prompt、AI 返回结果、执行状态和错误信息。

AI 返回 JSON 字段固定为：

```
{
  "category": "指定分析品类",
  "style_direction": "当前主流款式方向",
  "silhouette": "热门廓形版型",
  "core_structure": [],
  "color_palette": [],
  "fabric_feel": [],
  "design_keywords": []
}
```

2. MVP 功能范围

包含功能

- 前端填写趋势分析表单
- 后端创建趋势分析任务
- 后端组装固定 Prompt
- 后端调用 OpenAI 兼容 AI 大模型接口
- 后端解析并校验 AI 返回 JSON
- 后端保存分析记录
- 前端展示单条分析结果
- 前端展示历史分析记录列表
- 基础错误提示

不包含功能

- 登录和权限
- 支付
- 消息队列
- 微服务拆分

- 复杂任务调度
- 多模型管理后台
- 结果人工编辑和审核
- 文件上传

3. 推荐技术栈

前端

- Vue 3 或 React
- TypeScript
- Axios 或 Fetch
- Ant Design Vue、Element Plus、Ant Design React 等 UI 组件库

后端

- Node.js + NestJS / Express
- 或 Python + FastAPI
- 本文档示例以 Node.js 风格伪代码描述

数据库

- MySQL 8.x

AI 接口

- OpenAI 兼容 Chat Completions 接口
- `API Key`、`base_url`、`model` 从环境变量或配置文件读取

示例环境变量：

```
AI_API_KEY=your_api_key
AI_BASE_URL=https://api.example.com/v1
AI_MODEL=gpt-4o-mini
```

4. 业务流程

1. 用户进入“服装爆款趋势分析”页面。
2. 用户填写品类、目标人群、使用场景、风格方向。
3. 前端调用 `POST /api/trend-analysis`。
4. 后端校验请求参数。
5. 后端根据用户输入组装 Prompt。
6. 后端创建数据库记录，状态为 `processing`。
7. 后端调用 AI 大模型接口。
8. 后端获取 AI 返回内容。
9. 后端解析 AI 返回 JSON。
10. 后端校验 JSON 字段是否完整。
11. 校验成功后，更新数据库记录，状态为 `success`。
12. 校验失败或调用失败，更新数据库记录，状态为 `failed`，保存错误信息。
13. 前端展示分析结果或错误提示。
14. 用户可以查看单条结果或历史记录。

5. 接口设计

5.1 创建趋势分析

请求路径：

```
POST /api/trend-analysis
```

请求参数：

```
{
  "category": "连衣裙",
  "target_user": "25-35岁都市通勤女性",
  "scene": "春夏通勤、约会、轻商务",
  "style": "简约高级、法式优雅"
}
```

参数说明：

字段	类型	必填	说明
category	string	是	分析品类，例如连衣裙、女装外套、卫衣
target_user	string	是	目标人群
scene	string	是	使用场景
style	string	是	风格方向

成功返回示例：

```
{
  "id": 1,
  "status": "success",
  "result": {
    "category": "连衣裙",
    "style_direction": "简约高级与法式优雅结合，强调轻通勤和日常约会的多场景适配。",
    "silhouette": "收腰 A 字、直筒微宽松、轻伞摆廓形",
    "core_structure": [
      "收腰结构",
      "中长裙摆",
      "小 V 领或方领",
      "腰部可调节系带"
    ],
    "color_palette": [
      "奶油白",
      "浅卡其",
      "雾霾蓝",
      "鼠尾草绿"
    ],
    "fabric_feel": [
      "轻薄垂顺",
      "柔软亲肤",
      "微弹抗皱",
      "自然肌理感"
    ],
    "design_keywords": [
      "轻通勤",

```

```
        "法式收腰",
        "低饱和色",
        "一衣多穿",
        "精致松弛感"
    ]
}
}
```

失败返回示例:

```
{
  "id": 1,
  "status": "failed",
  "message": "AI 返回内容不是合法 JSON"
}
```

5.2 查询单条结果

请求路径:

```
GET /api/trend-analysis/{id}
```

成功返回示例:

```
{
  "id": 1,
  "category": "连衣裙",
  "target_user": "25-35岁都市通勤女性",
  "scene": "春夏通勤、约会、轻商务",
  "style": "简约高级、法式优雅",
  "result_json": {
    "category": "连衣裙",
    "style_direction": "简约高级与法式优雅结合，强调轻通勤和日常约会的多场景适配。",
    "silhouette": "收腰 A 字、直筒微宽松、轻伞摆廓形",
    "core_structure": ["收腰结构", "中长裙摆"],
    "color_palette": ["奶油白", "浅卡其"],
    "fabric_feel": ["轻薄垂顺", "柔软亲肤"],
    "design_keywords": ["轻通勤", "法式收腰"]
  },
  "status": "success",
  "error_message": null,
  "created_at": "2026-05-14 20:30:00",
  "updated_at": "2026-05-14 20:30:10"
}
```

5.3 查询历史记录

请求路径:

```
GET /api/trend-analysis
```

可选查询参数:

字段	类型	必填	说明
page	number	否	页码，默认 1
page_size	number	否	每页数量，默认 20

成功返回示例：

```
{
  "list": [
    {
      "id": 1,
      "category": "连衣裙",
      "target_user": "25-35岁都市通勤女性",
      "scene": "春夏通勤、约会、轻商务",
      "style": "简约高级、法式优雅",
      "status": "success",
      "created_at": "2026-05-14 20:30:00"
    }
  ],
  "pagination": {
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```

6. 数据库设计

表名：

```
trend_analysis_record
```

建表 SQL：

```
CREATE TABLE trend_analysis_record (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键 ID',
  category VARCHAR(100) NOT NULL COMMENT '品类',
  target_user VARCHAR(255) NOT NULL COMMENT '目标人群',
  scene VARCHAR(255) NOT NULL COMMENT '使用场景',
  style VARCHAR(255) NOT NULL COMMENT '风格方向',
  prompt TEXT NOT NULL COMMENT '组装后的 Prompt',
  result_json JSON NULL COMMENT 'AI 返回并校验后的 JSON 结果',
  status VARCHAR(20) NOT NULL DEFAULT 'processing' COMMENT '状态：
processing/success/failed',
  error_message TEXT NULL COMMENT '错误信息',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (id),
  INDEX idx_status (status),
  INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='服装爆款趋势分析记录表';
```

状态说明：

status	说明
processing	分析处理中
success	分析成功
failed	分析失败

7. 后端核心逻辑设计

7.1 Prompt 组装逻辑

固定 Prompt 模板：

你是一名资深服装趋势分析师，请根据用户输入，分析该服装品类在当前市场中的爆款趋势。

用户输入：

- 品类：{{category}}

- 目标人群：{{target_user}}

- 使用场景：{{scene}}

- 风格方向：{{style}}

请只返回标准 JSON，不要返回 Markdown，不要返回解释性文字。

JSON 字段必须严格如下：

{

"category": "指定分析品类",

"style_direction": "当前主流款式方向",

"silhouette": "热门廓形版型",

"core_structure": [],

"color_palette": [],

"fabric_feel": [],

"design_keywords": []

}

字段要求：

1. category 必须等于用户输入的品类。

2. style_direction 用一句话描述当前主流款式方向。

3. silhouette 用一句话描述热门廓形版型。

4. core_structure 返回 3-6 个核心结构要点。

5. color_palette 返回 3-6 个流行颜色。

6. fabric_feel 返回 3-6 个面料手感关键词。

7. design_keywords 返回 5-10 个设计关键词。

8. 所有内容使用中文。

Prompt 组装伪代码：

function buildTrendAnalysisPrompt(input) {

return `

你是一名资深服装趋势分析师，请根据用户输入，分析该服装品类在当前市场中的爆款趋势。

用户输入：

- 品类：\${input.category}

- 目标人群：\${input.target_user}

- 使用场景：\${input.scene}

- 风格方向：\${input.style}

请只返回标准 JSON，不要返回 Markdown，不要返回解释性文字。

JSON 字段必须严格如下：

```
{
  "category": "指定分析品类",
  "style_direction": "当前主流款式方向",
  "silhouette": "热门廓形版型",
  "core_structure": [],
  "color_palette": [],
  "fabric_feel": [],
  "design_keywords": []
}
```

字段要求：

1. **category** 必须等于用户输入的品类。
2. **style_direction** 用一句话描述当前主流款式方向。
3. **silhouette** 用一句话描述热门廓形版型。
4. **core_structure** 返回 3-6 个核心结构要点。
5. **color_palette** 返回 3-6 个流行颜色。
6. **fabric_feel** 返回 3-6 个面料手感关键词。
7. **design_keywords** 返回 5-10 个设计关键词。
8. 所有内容使用中文。

```
`;
}
```

7.2 AI 大模型调用逻辑

AI 调用要求：

- 使用 OpenAI 兼容接口
- `api_key` 从环境变量读取
- `base_url` 从环境变量读取
- `model` 从环境变量读取
- 设置较低 temperature，减少 JSON 格式漂移

伪代码：

```
async function callAiModel(prompt) {
  const response = await fetch(`${process.env.AI_BASE_URL}/chat/completions`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.AI_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      model: process.env.AI_MODEL,
      temperature: 0.3,
      messages: [
        {
          role: 'system',
          content: '你是一个严格返回 JSON 的服装趋势分析助手。'
        },
        {
          role: 'user',
          content: prompt
        }
      ]
    })
  })
}
```

```

    ]
  })
});

if (!response.ok) {
  throw new Error(`AI 接口调用失败: ${response.status}`);
}

const data = await response.json();
return data.choices[0].message.content;
}

```

7.3 JSON 解析与校验

校验规则：

- 返回内容必须能被 `JSON.parse` 解析
- 必须包含全部固定字段
- `core_structure` 必须是数组
- `color_palette` 必须是数组
- `fabric_feel` 必须是数组
- `design_keywords` 必须是数组
- `category` 建议等于用户输入的品种

伪代码：

```

function parseAndValidateAiResult(rawContent, input) {
  let result;

  try {
    result = JSON.parse(rawContent);
  } catch (error) {
    throw new Error('AI 返回内容不是合法 JSON');
  }

  const requiredFields = [
    'category',
    'style_direction',
    'silhouette',
    'core_structure',
    'color_palette',
    'fabric_feel',
    'design_keywords'
  ];

  for (const field of requiredFields) {
    if (!(field in result)) {
      throw new Error(`AI 返回缺少字段: ${field}`);
    }
  }

  const arrayFields = [
    'core_structure',
    'color_palette',
    'fabric_feel',
    'design_keywords'
  ];
}

```



```

];

for (const field of arrayFields) {
  if (!Array.isArray(result[field])) {
    throw new Error(`字段 ${field} 必须是数组`);
  }
}

if (result.category !== input.category) {
  result.category = input.category;
}

return result;
}

```

7.4 创建分析记录核心流程

伪代码：

```

async function createTrendAnalysis(input) {
  validateInput(input);

  const prompt = buildTrendAnalysisPrompt(input);

  const record = await db.trendAnalysisRecord.create({
    category: input.category,
    target_user: input.target_user,
    scene: input.scene,
    style: input.style,
    prompt,
    status: 'processing'
  });

  try {
    const rawContent = await callAiModel(prompt);
    const resultJson = parseAndValidateAiResult(rawContent, input);

    await db.trendAnalysisRecord.update(record.id, {
      result_json: resultJson,
      status: 'success',
      error_message: null
    });

    return {
      id: record.id,
      status: 'success',
      result: resultJson
    };
  } catch (error) {
    await db.trendAnalysisRecord.update(record.id, {
      status: 'failed',
      error_message: error.message
    });

    return {
      id: record.id,
      status: 'failed',

```

```
      message: error.message
    };
  }
}
```

8. 前端页面设计

页面模块

- 1. 输入表单区
- 2. 提交按钮区
- 3. 分析结果展示区
- 4. 历史记录区

表单字段

字段	组件	示例
品类	输入框	连衣裙
目标人群	输入框	25-35岁都市通勤女性
使用场景	输入框或文本域	春夏通勤、约会、轻商务
风格方向	输入框或文本域	简约高级、法式优雅

页面交互

- 用户填写字段后点击“生成趋势分析”
- 提交中按钮显示 loading
- 成功后展示结构化结果
- 失败后展示错误信息
- 页面下方展示历史记录
- 点击历史记录可查看单条详情

结果展示建议

可按以下结构展示：

- 指定分析品类
- 当前主流款式方向
- 热门廓形版型
- 核心结构
- 流行色彩
- 面料手感
- 设计关键词

9. 错误处理

前端错误处理

- 必填字段为空：提示用户补充完整
- 接口请求失败：提示“生成失败，请稍后重试”
- AI 分析失败：展示后端返回的错误信息
- 历史记录为空：展示空状态

后端错误处理

场景	处理方式
参数缺失	返回 400
参数长度过长	返回 400
AI 接口调用失败	记录 failed 状态和错误信息
AI 返回非 JSON	记录 failed 状态和错误信息
AI 返回字段缺失	记录 failed 状态和错误信息
数据库异常	返回 500

参数校验建议：

```
function validateInput(input) {
  const fields = ['category', 'target_user', 'scene', 'style'];

  for (const field of fields) {
    if (!input[field] || typeof input[field] !== 'string') {
      throw new Error(`${field} 为必填字段`);
    }

    if (input[field].length > 255) {
      throw new Error(`${field} 长度不能超过 255 个字符`);
    }
  }
}
```

10. 推荐目录结构

```
project-root/
  frontend/
    src/
      api/
        trendAnalysis.ts
      pages/
        TrendAnalysisPage.vue
      components/
        TrendAnalysisForm.vue
        TrendAnalysisResult.vue
        TrendAnalysisHistory.vue
  backend/
    src/
      config/
        ai.config.ts
```

```
database.config.ts
modules/
  trend-analysis/
    trend-analysis.controller.ts
    trend-analysis.service.ts
    trend-analysis.repository.ts
    trend-analysis.dto.ts
    trend-analysis.types.ts
  utils/
    json.ts
  app.ts
migrations/
  create_trend_analysis_record.sql
.env
```

11. MVP 开发顺序

1. 创建 MySQL 表 `trend_analysis_record`
2. 搭建后端项目和数据库连接
3. 实现趋势分析记录 Repository
4. 实现 Prompt 组装函数
5. 实现 AI 大模型调用函数
6. 实现 JSON 解析与校验函数
7. 实现 `POST /api/trend-analysis`
8. 实现 `GET /api/trend-analysis/{id}`
9. 实现 `GET /api/trend-analysis`
10. 搭建前端页面
11. 实现前端表单提交
12. 实现分析结果展示
13. 实现历史记录列表
14. 联调完整流程
15. 补充基础测试用例

12. 测试用例

12.1 创建趋势分析成功

输入：

```
{
  "category": "连衣裙",
  "target_user": "25-35岁都市通勤女性",
  "scene": "春夏通勤、约会、轻商务",
  "style": "简约高级、法式优雅"
}
```

预期：

- 接口返回 `status = success`
- 返回结果包含全部固定字段
- 数据库保存用户输入
- 数据库保存 Prompt
- 数据库保存 `result_json`

12.2 参数缺失

输入:

```
{
  "category": "连衣裙",
  "target_user": "",
  "scene": "春夏通勤",
  "style": "法式优雅"
}
```

预期:

- 接口返回 400
- 提示 `target_user` 为必填字段

12.3 AI 返回非 JSON

模拟 AI 返回:

这是一个趋势分析结果，不是 JSON。

预期:

- 接口返回 `status = failed`
- 数据库记录状态为 `failed`
- `error_message` 保存“AI 返回内容不是合法 JSON”

12.4 AI 返回字段缺失

模拟 AI 返回:

```
{
  "category": "连衣裙",
  "style_direction": "简约通勤方向"
}
```

预期:

- 接口返回 `status = failed`
- 数据库记录状态为 `failed`
- `error_message` 保存缺失字段名称

12.5 查询单条结果

请求:

```
GET /api/trend-analysis/1
```

预期:

- 返回 id 为 1 的分析记录
- 包含用户输入、结果 JSON、状态、创建时间

12.6 查询历史记录

请求：

```
GET /api/trend-analysis?page=1&page_size=20
```

预期：

- 返回列表数据
- 返回分页信息
- 按创建时间倒序排列

13. 备注

MVP 阶段重点是打通“输入参数 -> 组装 Prompt -> 调用 AI -> 校验 JSON -> 保存记录 -> 展示结果”的完整闭环。后续可以再扩展趋势来源、平台数据、图片生成、款式图推荐、报告导出等能力。